CS 3244

Machine Learning

Week 11 Pre-Class
Dimensionality Reduction and Clustering

11 pre

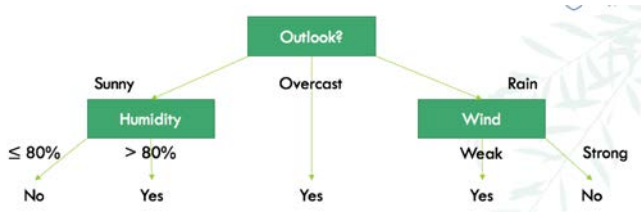
Kan Min-Yen
Quoc Trung Luong

Background Photo credits: Rafiq Mirza, Luan Ahn, Rosmarie Voegtli @ Flickr

Recap and Forecast

Decision Tree Learning (DTL)

- Expressive: can overfit to achieve $J_{train} = 0$
- Need pruning to combat variance



Extensions: Diversity is good

Bootstrap & Bagging: Sample with replacement to generate new $\widetilde{\mathcal{D}}_t$.

- With each, train a h_t, apply uniform voting.
- Variance lowered through averaging.

Boosting: Make a strong classifier from simple weak classifiers.

- Reweight $\widetilde{\mathcal{D}}_t$ to account for easy / hard examples.
- Lower bias via simple hypotheses.

Random Forest: Combine *Bagging* with DTL and introduce feature projection.

- Each h_t tuned to some subset of the features.

Learning Outcomes for this week:

- Have a basic understanding of unsupervised learning;
- Understand different unsupervised learning algorithms for:
  - Dimensionality reduction: For aiding visualization, exploratory analysis;
  - Clustering: Finding latent patterns.
- Implement and understand k means clustering;
- Understand the Expectation Maximization (EM) meta-algorithm;
- Apply unsupervised learning on real-world data.

Dimensionality Reduction

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Big Data

Input data may have thousands or millions of dimensions!

- e.g., facial images, natural language text, stock market time series

Can we describe them more succinctly? That would help…

- easier learning – fewer parameters
- visualization – hard to visualize more than 3D or 4D
- discover "intrinsic dimensionality" of data
  - high dimensional data that is truly lower dimensional

Dimensionality reduction

Example: Stock market data

Presence of inherent structure:

- Certain equities might change in a similar way
  - E.g., Transshipment stocks, Regional stocks
- Use less information to get a good approximation

Photo credit: Andreas Poike @ Flickr

Lower dimensional projections

Rather than picking a subset of the features, we can construct new features that are combinations of existing features.

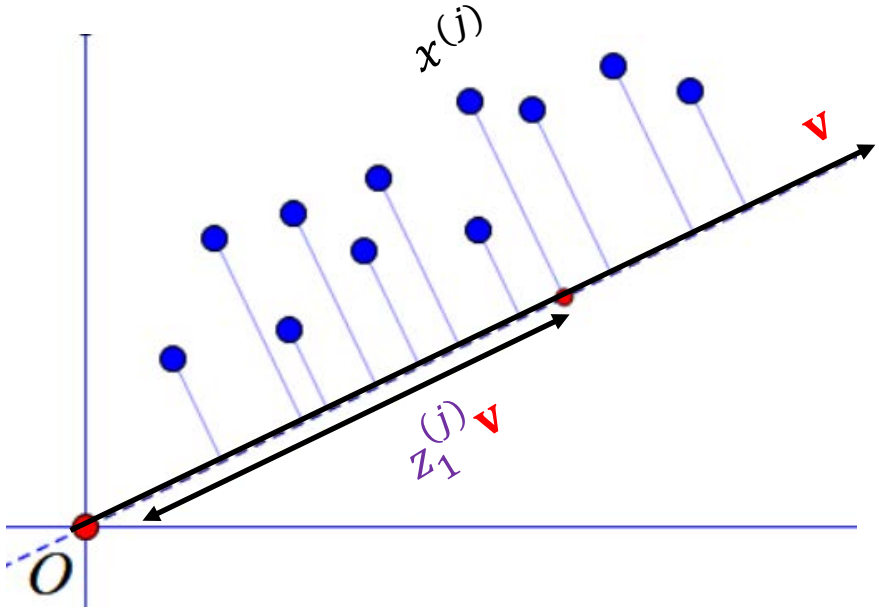
Let's see this in the unsupervised setting (just $\mathbf{X}$, drop y).

Same for all points

Closest point on $\mathbf{v}$ to original $\mathbf{x}$

Example: $\mathbf{x}^{(j)}: \{x_1, x_2\} \rightarrow \Phi_{\mathbf{v}}(\mathbf{x}): \{z_1^{(j)}\}$

Linear function $\Phi_{\mathbf{v}}: \{x_1, x_2\} = z_1 \cdot \mathbf{v} = z_1 \cdot \{v_1, v_2\}$



Diagrams modified from Alex Ihler's slides @ UCI

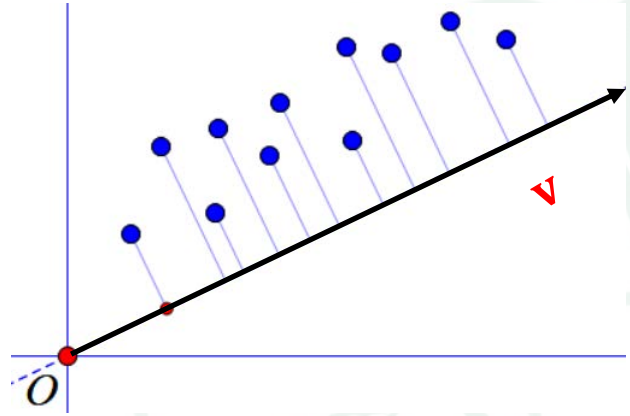
Let's Analyze a 2D example

Original 2D points: $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n)}\}$

Project on $\mathbf{v}$, get 1D points:

$$\{z_1^{(1)}, \dots, z_1^{(n)}\}$$

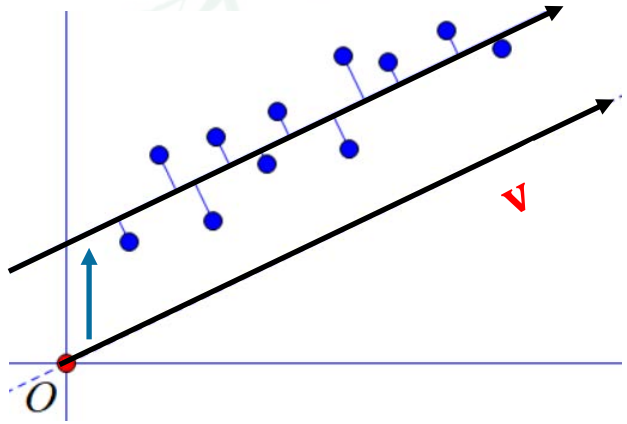
* *The reference point could be any point on* $\mathbf{v}$



By projection, we can lose some information.

Reconstruct the original data by shifting $\mathbf{v}$ so that the sum of squares of the distance from the original data point is minimized.

Direction that leads to smallest error is most desirable.

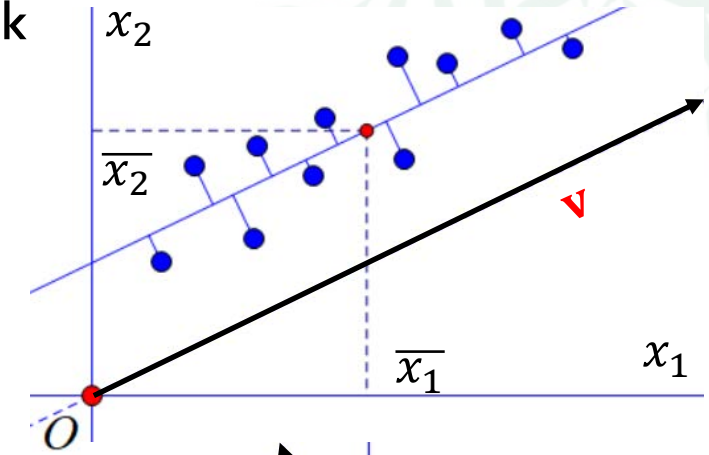


Observation

1. The closest line should pass through center of the points

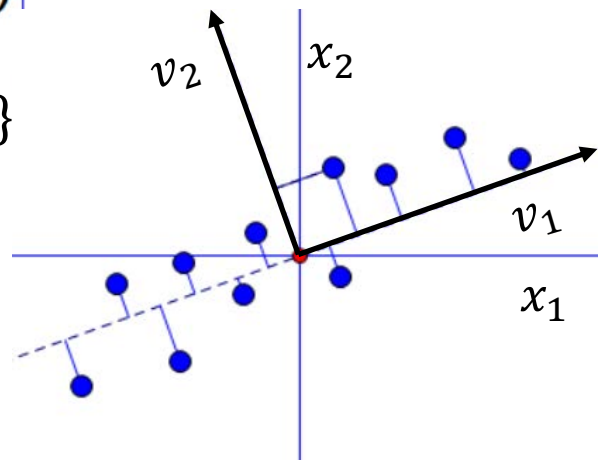
   → Conveniently reconstruct original points by demean them, project to $\mathbf{v}$ then add $\bar{\mathbf{x}}$ back

   → Reconstruction point: $\mathbf{x}_{rec}^{(j)} = \bar{\mathbf{x}} + z^{(j)} \cdot \mathbf{v}$



2. The procedure can be viewed as:
   - Transform to new coordinate $\{x_1, x_2\} \rightarrow \{v_1, v_2\}$ (The new origin is at the center of the points)
   - Drop unimportant dimensions $\{v_1, v_2\} \rightarrow \{v_1\}$
   - Reconstruct by:

$$\{v_1\} \rightarrow \{v_1, 0\} \xrightarrow{Inverse\ Transform} \{x_1, x_2\}$$



Linear projection

Project a point into a another space:

Row vectors

1. Point: $\mathbf{x} = \{x_1, \dots, x_n\}$
2. Select a basis: a set of basis vectors: $\{\mathbf{v}_1, \dots, \mathbf{v}_N\}$
3. Let's consider only orthonormal bases: e.g., $\mathbf{v}_i \mathbf{v}_i^\top = 1$ and $\mathbf{v}_i \mathbf{v}_j^\top = 0$ for $i \neq j$
4. Select a center: $\bar{\mathbf{x}}$ defines the offset of the space
5. Represent $\mathbf{x}$ in new basis: $\{z_1, \dots, z_N\}$ where $z_k = (\mathbf{x} - \bar{\mathbf{x}})\mathbf{v}_k^\top$
6. Projected point: $\{z_1, \dots, z_K\}$

PCA slides largely based on Guestrin @ CMU's slides.

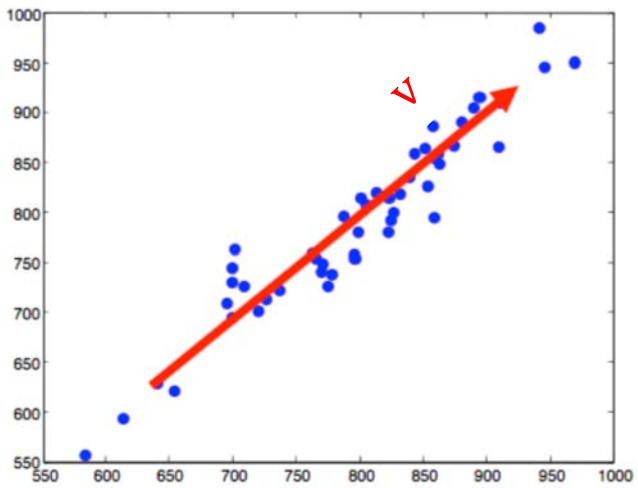
Principal component analysis

Project n-dimensional data into K-dimensional space while preserving information:

- e.g., project space of 30,000 words into 50-dimensions
- e.g., project 2-d into 1-d

Compute projection $\mathbf{v}$ with minimum (OLS) reconstruction error:

$$\min \sum_{j=1}^{m} \left\| \mathbf{x}^{(j)} - \hat{\mathbf{x}}^{(j)} \right\|^2 = \min \sum_{j=1}^{m} \left\| \mathbf{x}^{(j)} - (\bar{\mathbf{x}} + \sum_{k=1}^{K} z_k^{(j)} \mathbf{v}_k) \right\|^2$$

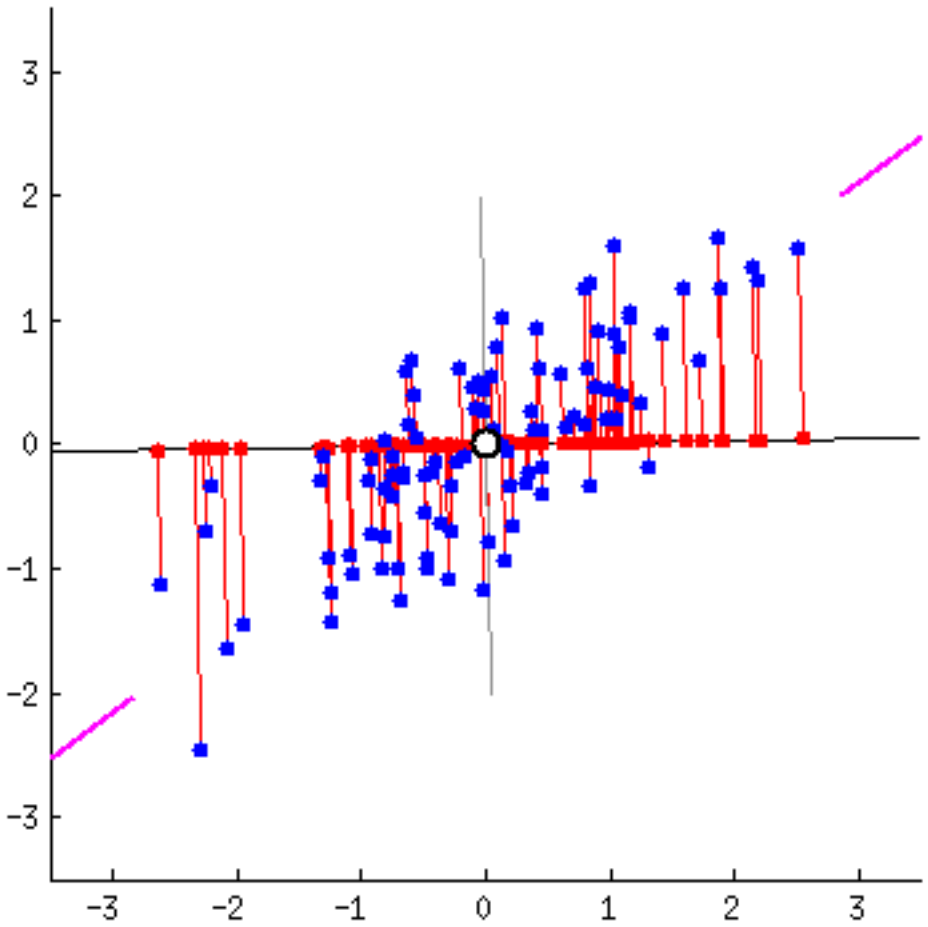


Choose $\mathbf{v}$ to minimize residual variance

$\mathbf{v}$ is thus also the direction of maximum variance

Diagrams modified from Alex Ihler's slides @ UCI

Choosing representative axes



Reconstruction Cost J_{recon}

A point is exactly represented by:

$$\mathbf{x}^{(j)} = \bar{\mathbf{x}} + \sum_{k=1}^{n} z_k^{(j)} \mathbf{v}_k$$

But in PCA, we only reconstruct with $K < n$ dimensions:

$$\hat{\mathbf{x}}^{(j)} = \bar{\mathbf{x}} + \sum_{k=1}^{K} z_k^{(j)} \mathbf{v}_k$$

The cost then is: $J_{recon} = \sum_{j=1}^{m} \left\| \mathbf{x}^{(j)} - \hat{\mathbf{x}}^{(j)} \right\|^2$

$$\underset{(m \times n)}{\mathbf{X}} = \begin{bmatrix} \dots & \mathbf{x}^{(1)} & \dots \\ \dots & \mathbf{x}^{(j)} & \dots \\ \dots & \mathbf{x}^{(m)} & \dots \end{bmatrix}$$

Reconstruction Cost J_{recon}

A point is exactly represented by:

$$\mathbf{x}^{(j)} = \bar{\mathbf{x}} + \sum_{k=1}^{n} z_k^{(j)} \mathbf{v}_k$$

But in PCA, we only reconstruct with $K < n$ dimensions:

$$\hat{\mathbf{x}}^{(j)} = \bar{\mathbf{x}} + \sum_{k=1}^{K} z_k^{(j)} \mathbf{v}_k$$

$$\underset{(m \times n)}{X} = \begin{bmatrix} \dots & \mathbf{x}^{(1)} & \dots \\ \dots & \mathbf{x}^{(j)} & \dots \\ \dots & \mathbf{x}^{(n)} & \dots \end{bmatrix}$$

The cost then is: $J_{recon} = \sum_{j=1}^{m} \left\| \mathbf{x}^{(j)} - \hat{\mathbf{x}}^{(j)} \right\|^2$

$$= \sum_{j=1}^{m} \left\| \left(\bar{\mathbf{x}} + \sum_{k=1}^{n} z_k^{(j)} \mathbf{v}_k \right) - \left(\bar{\mathbf{x}} + \sum_{k=1}^{K} z_k^{(j)} \mathbf{v}_k \right) \right\|^2$$

$$= \sum_{j=1}^{m} \left\| \left(\sum_{k=K+1}^{n} z_k^{(j)} \mathbf{v}_k \right) \right\|^2 = \sum_{j=1}^{m} \left[\left(\sum_{k=K+1}^{n} z_k^{(j)} \underbrace{\mathbf{v}_k \mathbf{v}_k^\top}_{1} z_k^{(j)} \right) + \sum_{\substack{K+1 \le h, l \le n \\ h \ne l}} z_h^{(j)} \underbrace{\mathbf{v}_h \mathbf{v}_l^\top}_{0} z_l^{(j)} \right]$$

Reconstruction Cost J_{recon}

$$J_{recon} = \sum_{j=1}^{m}\left[\left(\sum_{k=K+1}^{n} z_k^{(j)} \underbrace{\mathbf{v}_k\mathbf{v}_k^\top}_{1} z_k^{(j)}\right) + \sum_{\substack{K+1\le h,\, l\le n\\ h\neq l}} z_h^{(j)} \underbrace{\mathbf{v}_h\mathbf{v}_l^\top}_{0} z_l^{(j)}\right]$$

$$= \sum_{j=1}^{m}\sum_{k=K+1}^{n} z_k^{(j)^2} = \sum_{k=K+1}^{n}\sum_{j=1}^{m} z_k^{(j)^2}$$

$$\underset{(m\times n)}{\bar{\mathbf{X}}} = \begin{bmatrix} \dots & \bar{\mathbf{x}} & \dots \\ \dots & \bar{\mathbf{x}} & \dots \\ \dots & \bar{\mathbf{x}} & \dots \end{bmatrix}$$

$$= \sum_{k=K+1}^{n}\sum_{j=1}^{m} \mathbf{v}_k(\mathbf{x}-\bar{\mathbf{x}})^\top(\mathbf{x}-\bar{\mathbf{x}})\mathbf{v}_k^\top$$

Covariance matrix

$$= \sum_{k=K+1}^{n} \mathbf{v}_k(\mathbf{X}-\bar{\mathbf{X}})^\top(\mathbf{X}-\bar{\mathbf{X}})\,\mathbf{v}_k^\top = m\sum_{k=K+1}^{n} \mathbf{v}_k\Sigma\mathbf{v}_k^\top$$

This choice of v_k leads to a minimum J_{recon}

Let $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ be eigenvectors of Σ and the corresponding eigenvalues be $\{\lambda_1, \dots, \lambda_n\}$.

$$J_{recon} = m\sum_{k=K+1}^{n} \mathbf{v}_k\,\lambda_k\mathbf{v}_k^\top = m\sum_{k=K+1}^{n}\lambda_k$$

Better choose $\{\mathbf{v}_1, \dots, \mathbf{v}_K\}$ corresponding to highest eigenvalues

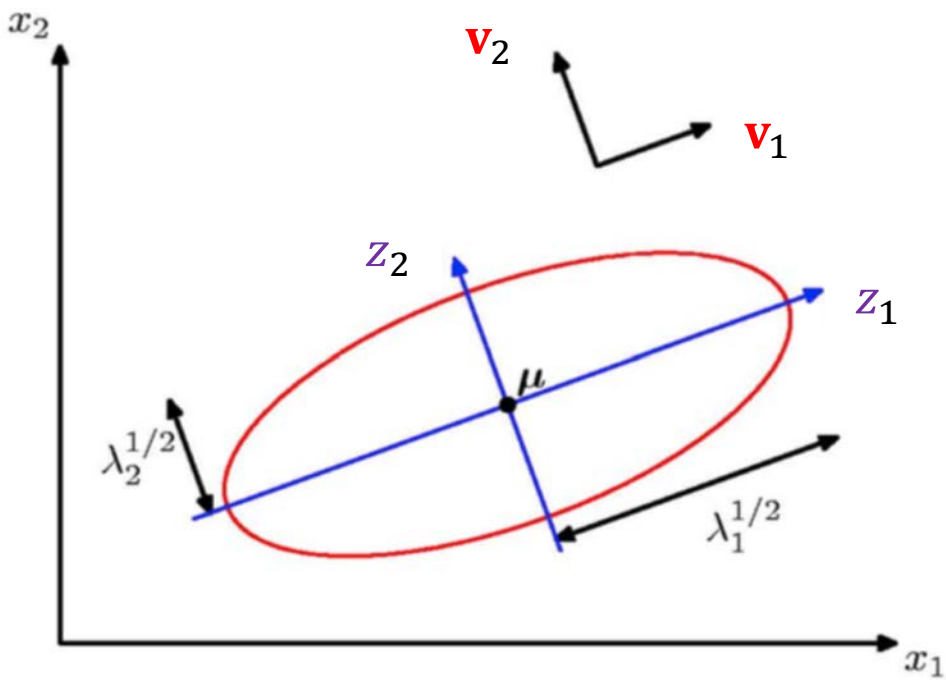
Back to the Gaussian…

$$\Delta^2 = (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu)$$

(Eigen)decompose covariance matrix $\Sigma = \mathrm{V}\Lambda\mathrm{V}^\top$

Write Σ in terms of eigenvectors:

$$\Sigma^{-1} = \Sigma_{j=1}^{n} \frac{1}{\lambda_j} \mathbf{v}_j \mathbf{v}_j^\top$$



Diagrams modified from Alex Ihler's slides @ UCI

Basic PCA algorithm

1. Start from $m \times n$ data matrix $\mathbf{X}$
2. Center: subtract μ from each row of $\mathbf{X}_c \leftarrow \mathbf{X} - \overline{\mathbf{X}}$
3. (Typically) scale each dimension by its variance

   To decorrelate the magnitude of the variable
4. Compute covariance matrix: $\Sigma \leftarrow \frac{1}{m}\mathbf{X}_c^\top \mathbf{X}_c$
5. Find eigenvectors and eigenvalues of Σ
6. Compute K eigenvectors with highest eigenvalues

   $\Sigma = \mathbf{V}\Lambda\mathbf{V}^\top$

Problem: can be slow when $\mathbf{X}$ is big

Singular Value Decomposition

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Singular Value Decomposition

One method to calculate the orthogonal decomposition of $\mathbf{X}$ in PCA.

Nice as we can obtain the K largest directly.

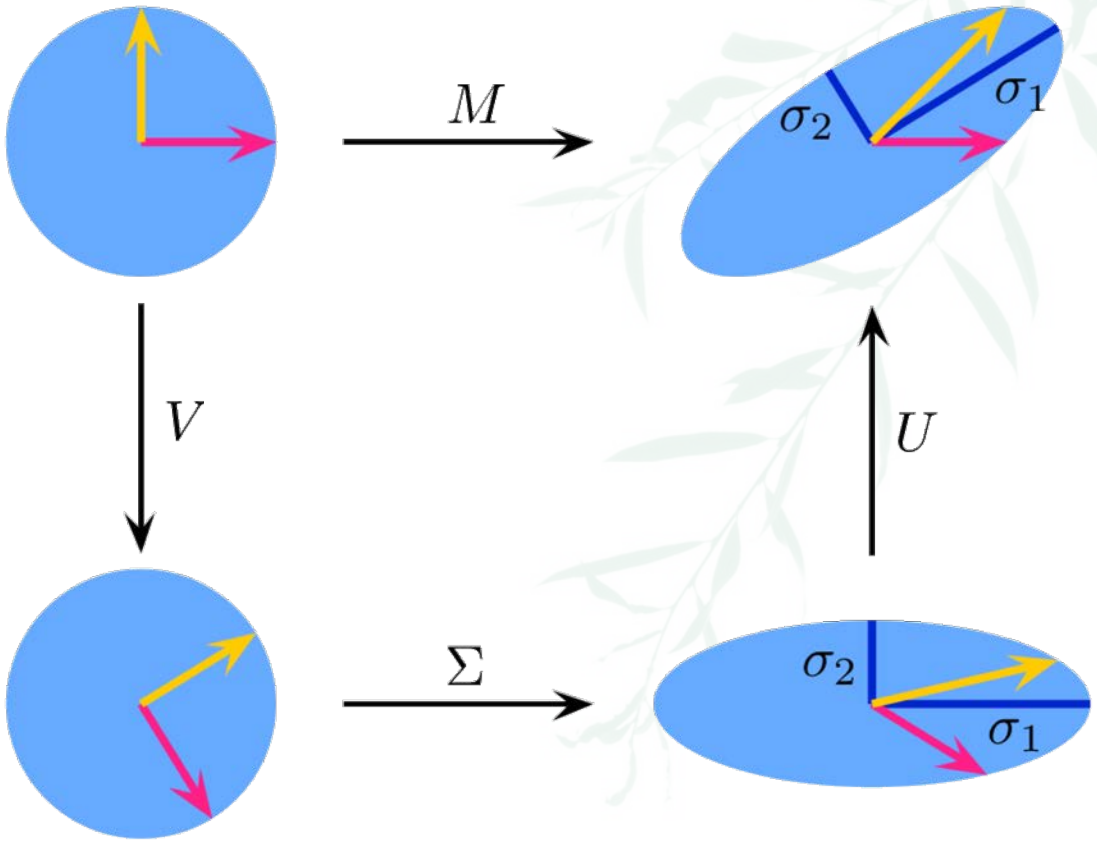
Nice because <u>all</u> matrices work with SVD.

- Other factorizations don't work with certain matrices (e.g., non-square).

Geometry of SVD

"A singularly valuable decomposition" – Dan Kalman

*Any transformation (matrix) can be thought of as a rotation V, followed by scaling Σ, followed by rotation U.**



$$M = U \cdot \Sigma \cdot V$$

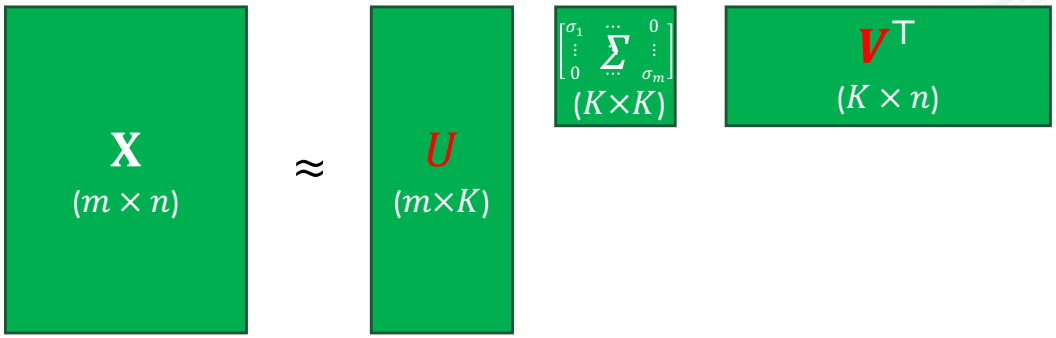
* *For square matrices only; the rotation interpretation doesn't work for rectangular matrices.*

By Georg-Johann - Own work, CC BY-SA 3.0, https://commons.wikimedia.org/w/index.php?curid=11342212

Singular Value Decomposition

Write $\mathbf{X} = U \Sigma V^\top$.

- U: weight matrix, one row per point – coordinate of $\mathbf{x}^{(j)}$ in eigenspace
- Σ: singular value matrix, a diagonal matrix
  - each entry is an eigenvalue λ_k
- $V^\top$: singular vector matrix
  - each row is an eigenvector $\mathbf{v}_k$

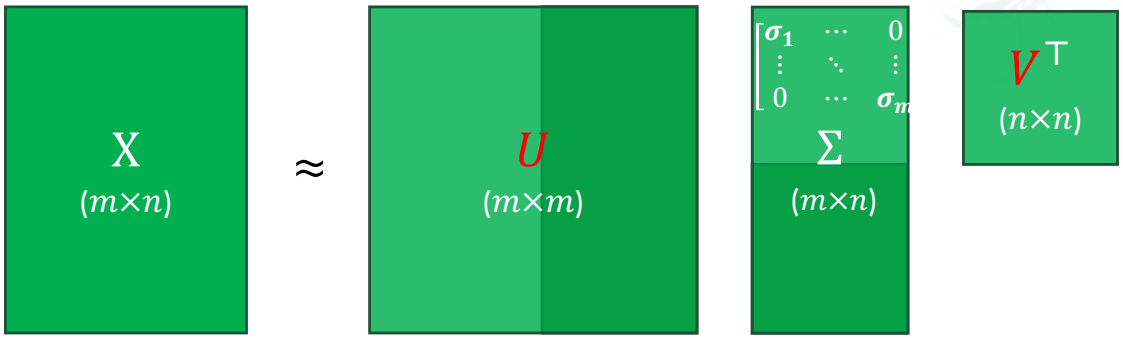


Singular Value Decomposition

Write $\mathbf{X} = U\,\Sigma\,V^\top$.

Orthogonal: $\mathbf{X}^\top\mathbf{X} = V\,\Sigma\,\Sigma\,V^\top = V\,D\,V^\top$
$\mathbf{X}\mathbf{X}^\top = U\,\Sigma\,\Sigma\,U^\top = U\,D\,U^\top$

- U: weight matrix, one row per point – coordinate of $\mathbf{x}^{(j)}$ in eigenspace
- Σ: singular value matrix, a diagonal matrix
  - each entry is an eigenvalue λ_k
- $V^\top$: singular vector matrix
  - each row is an eigenvector $\mathbf{v}_k$



PCA using SVD algorithm

1. Start from $m \times n$ data matrix $\mathbf{X}$
2. Re-center: subtract μ from each row of $\mathbf{X}_c \leftarrow \mathbf{X} - \bar{\mathbf{X}}$
3. (Typically) scale each dimension by its variance
   - To decorrelate the magnitude of variable
4. Call SVD algorithm on $\mathbf{X}_c$ – ask for K singular vectors

Principal components: K singular vectors with highest singular values (rows of $V^\top$)

- Coefficients are derived from $U\Sigma$ matrix:

$$\mathbf{x}^{(j)} = U_{j,1}\Sigma_{11}\mathbf{v}_1 + U_{j,2}\Sigma_{22}\mathbf{v}_2 + \cdots + U_{j,K}\Sigma_{KK}\mathbf{v}_K$$

PCA: Eigenfaces

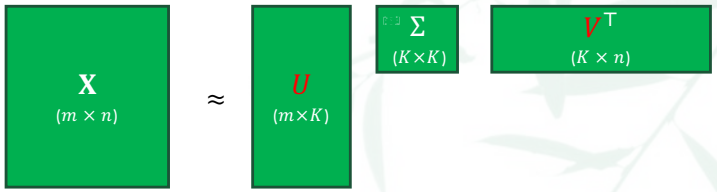
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

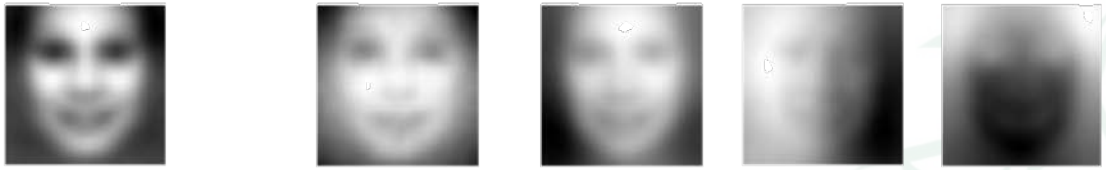
Eigenfaces

Eigen-$\mathbf{X}$ paradigm = represent $\mathbf{X}$ using PCA

"What about faces?" (Turk and Pentland, 91)

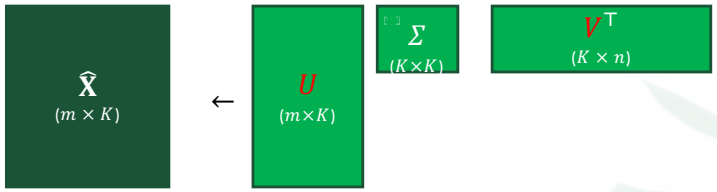


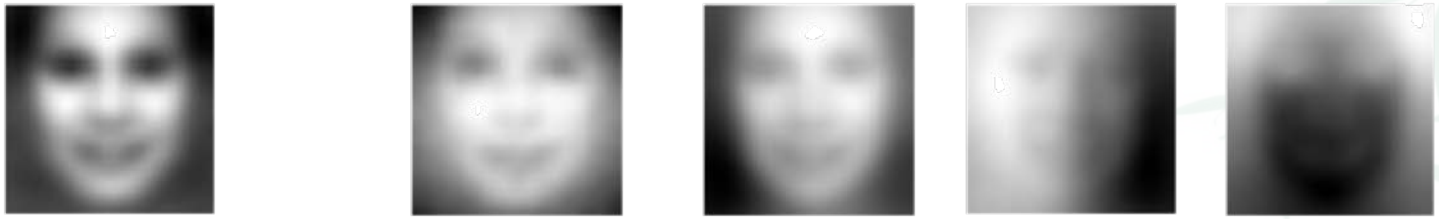
Decomposition:



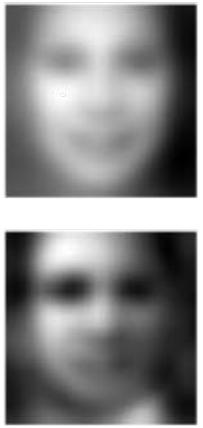
μ v_1 v_2 v_3 v_4 ...

Eigenfaces: Reconstruction



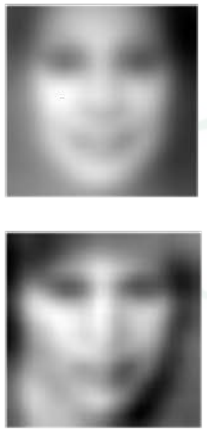


μ v_1 v_2 v_3 v_4 ...



$K = 4$

$K = 50$



$K = 4$

$K = 50$

SVD and PCA relationship

When the data is centered (mean 0) then PCA via SVD on all n dimensions yields the same result.

We typically want to do dimensionality reduction, so we choose $K \ll n$.

So no need to compute the full covariance matrix.

We also rarely have a square data matrix $\mathbf{X}$.

$\therefore$ Use SVD all the time!

K Means Clustering

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Clustering

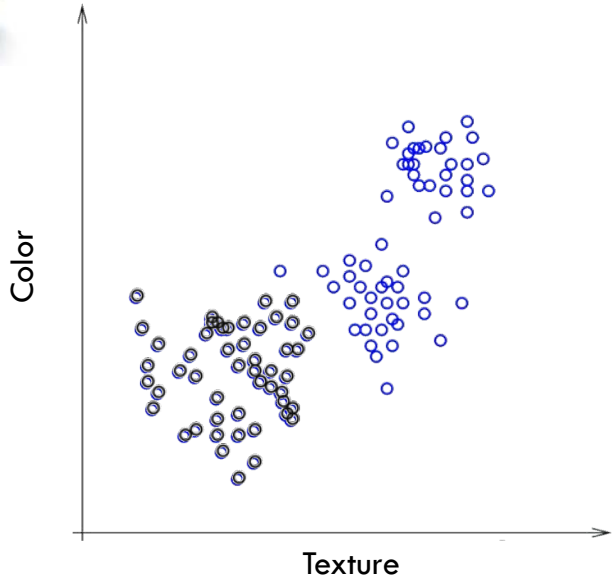


Photo credits: The Globe and Mail, annaterrenal @ DeviantArt,

Clustering

A cluster is a collection of points S.

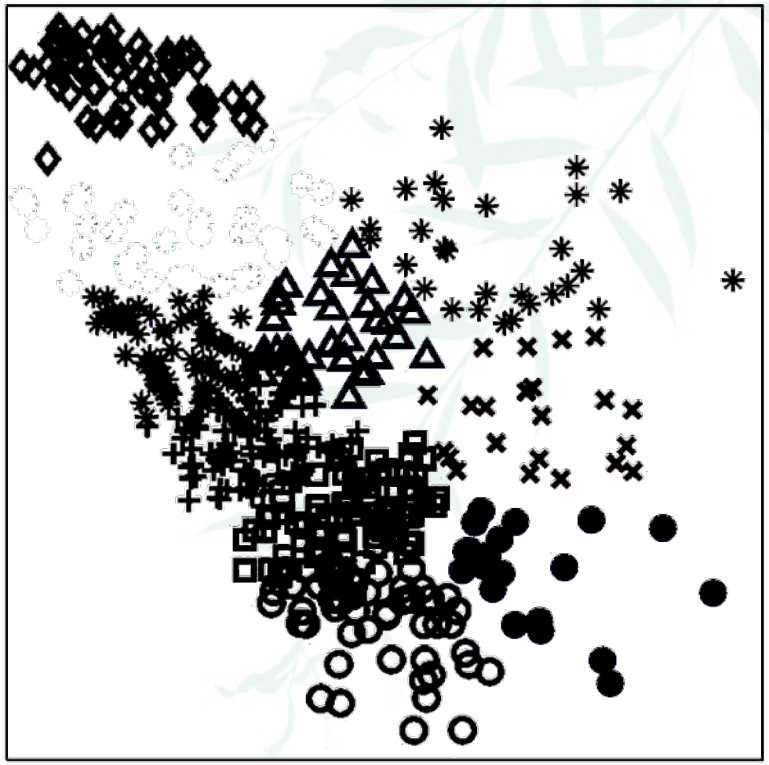
A K-clustering is a partition of the data into K clusters $S_1, S_2, \ldots, S_K$.

$$\bigcup_{k=1}^{K} S_k = \mathcal{D}$$

Hard clustering assigns each point into exactly one cluster:

$$S_i \cap S_j = \emptyset, \text{ for } i \neq j.$$

Each cluster has a center μ_k.



Clustering Quality

Points in a cluster should be similar (close to each other, and to the center)

Defines the error measure as well.

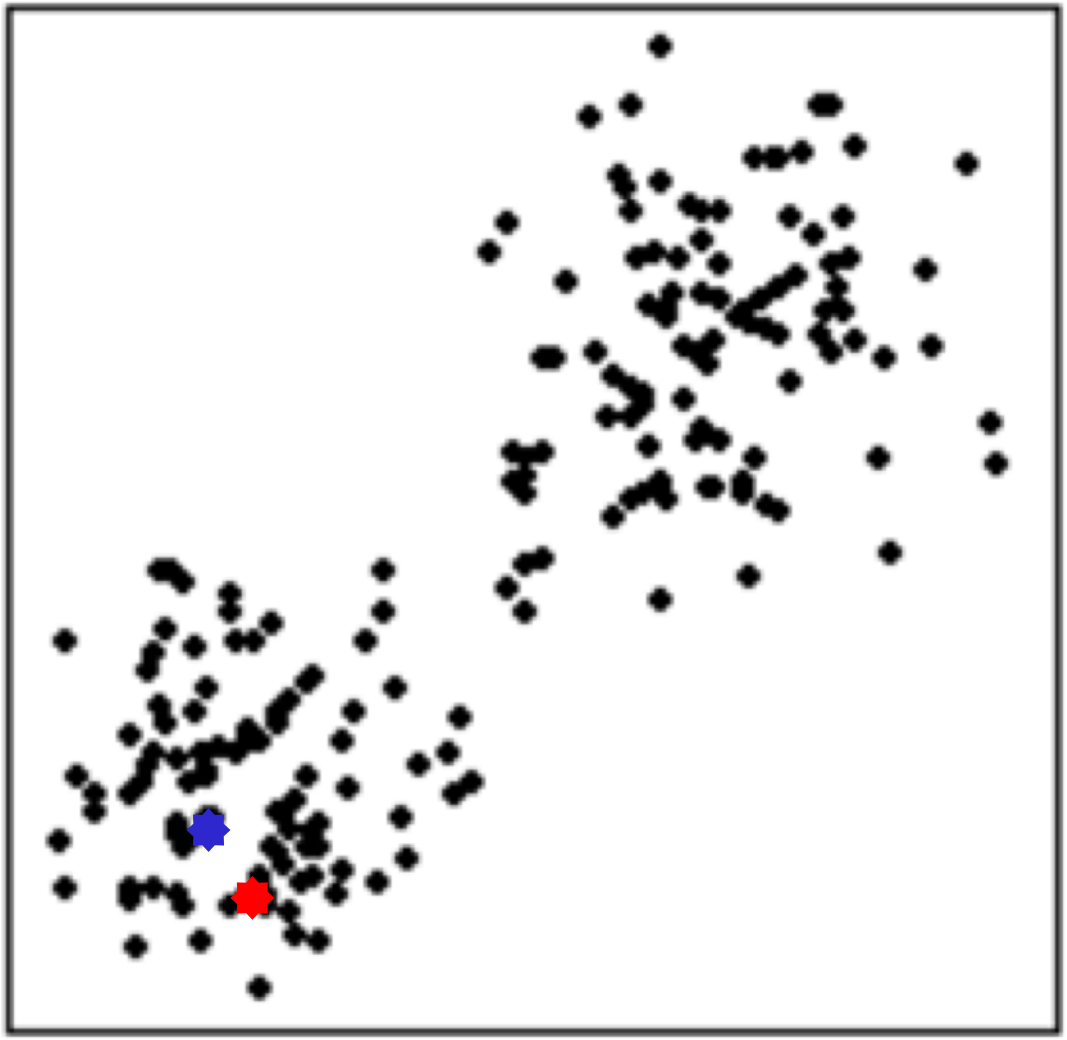
Error in cluster k: $J_k = \sum_{\mathbf{x}^{(j)} \in S_k} \left|\left|\mathbf{x}^{(j)} - \mu_k\right|\right|^2$.

Other measures (e.g., L^p norms) can be used as well

Minimizing $J_{clust} = \sum_{k=1}^{K} J_k$ is not convex, known to be NP hard.

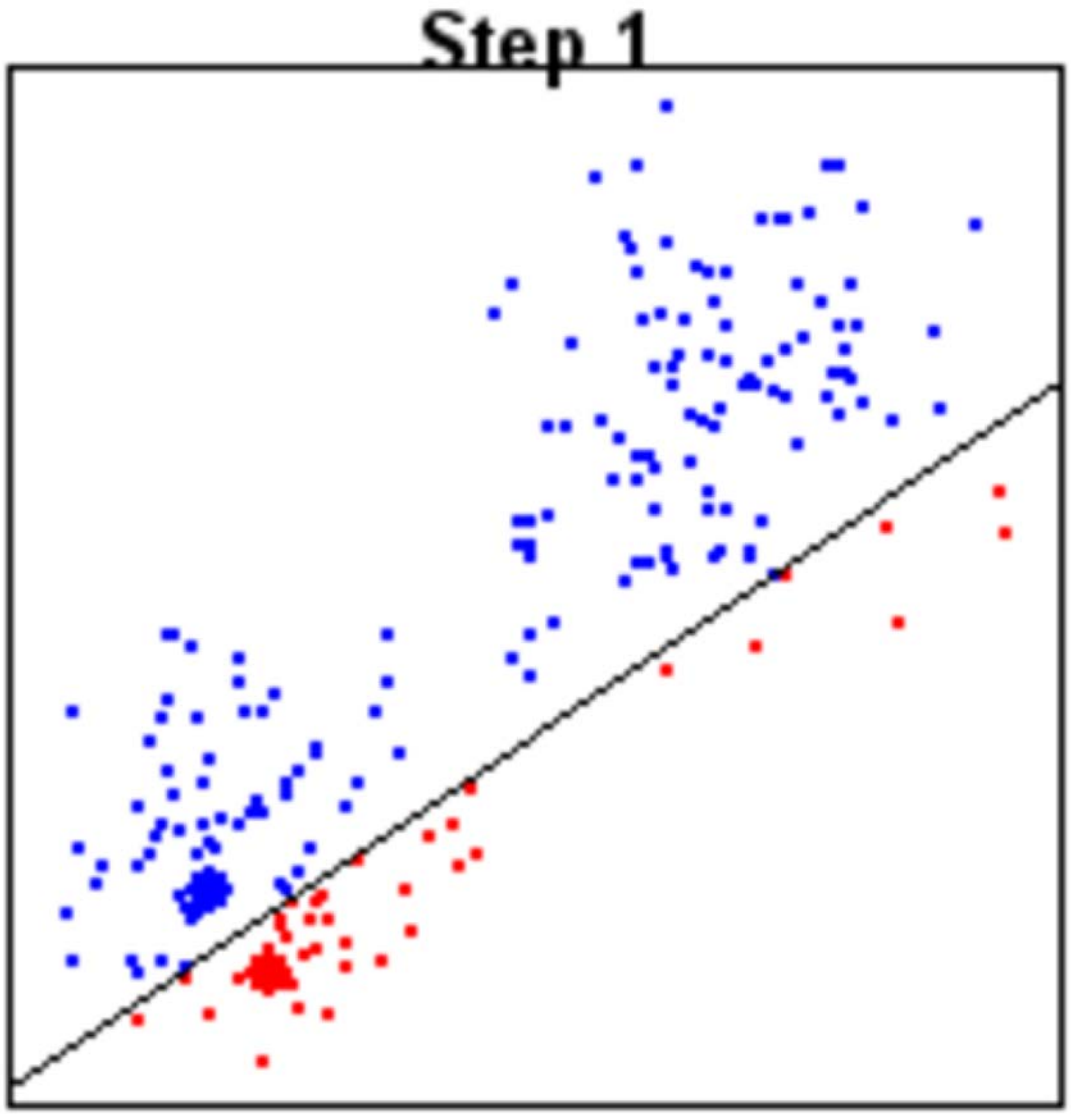
But simple, good solutions exist.

K means, Illustrated



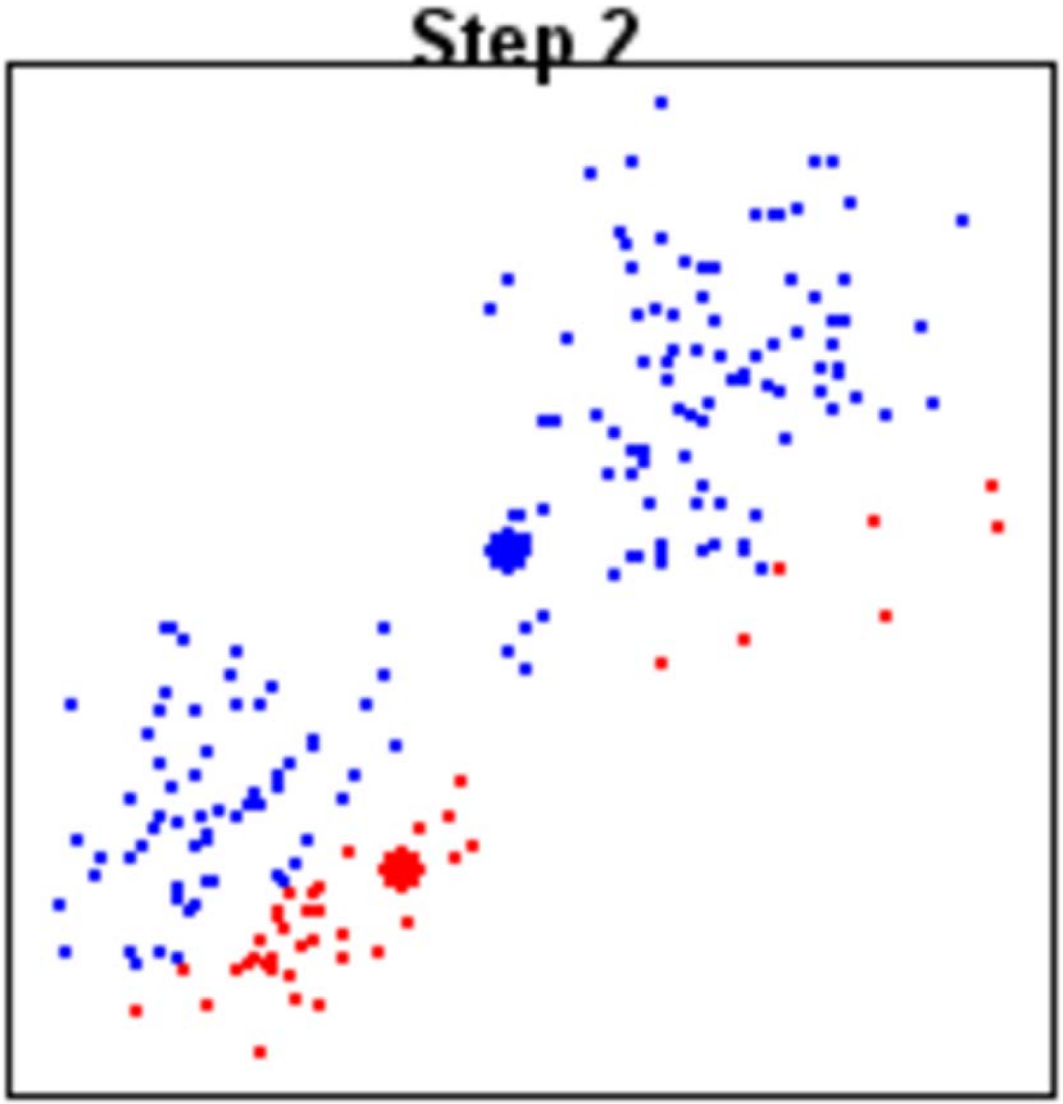
This slide's graphics are from PSU Summer School in Statistics for Astronomers

Assign points to clusters



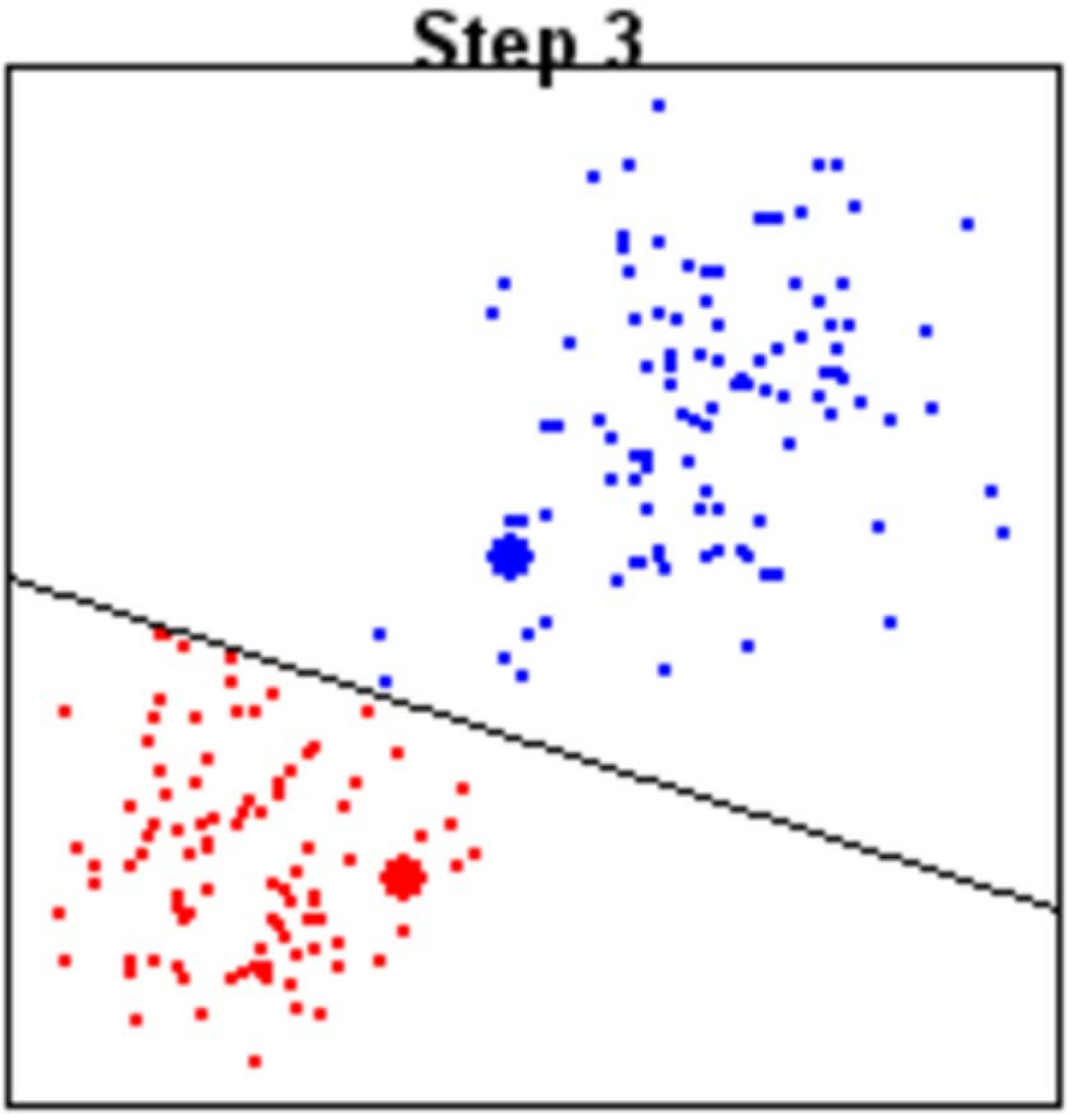
This slide's graphics are from PSU Summer School in Statistics for Astronomers

Recalculate cluster centers



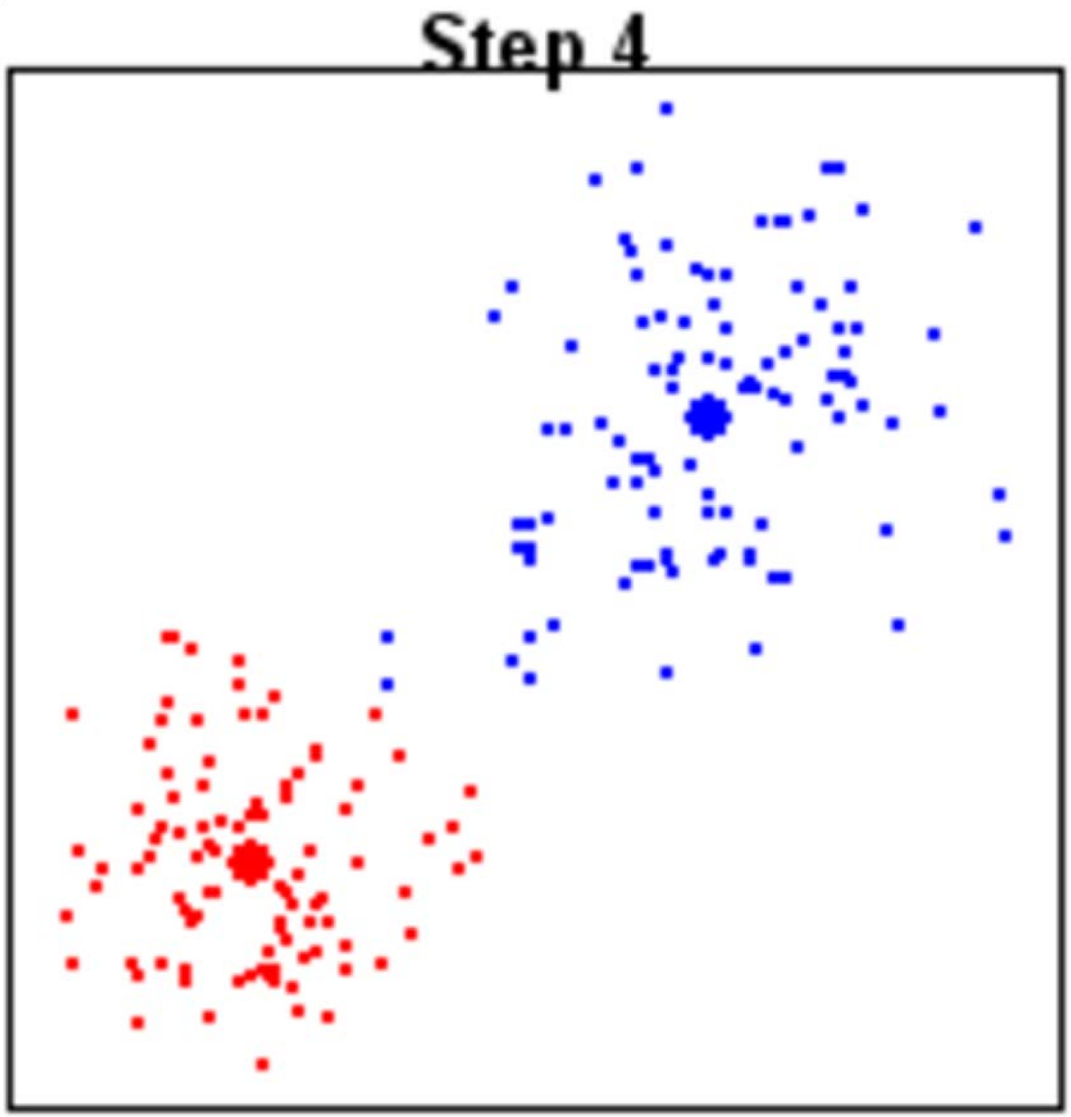
This slide's graphics are from PSU Summer School in Statistics for Astronomers

Reassign points to clusters



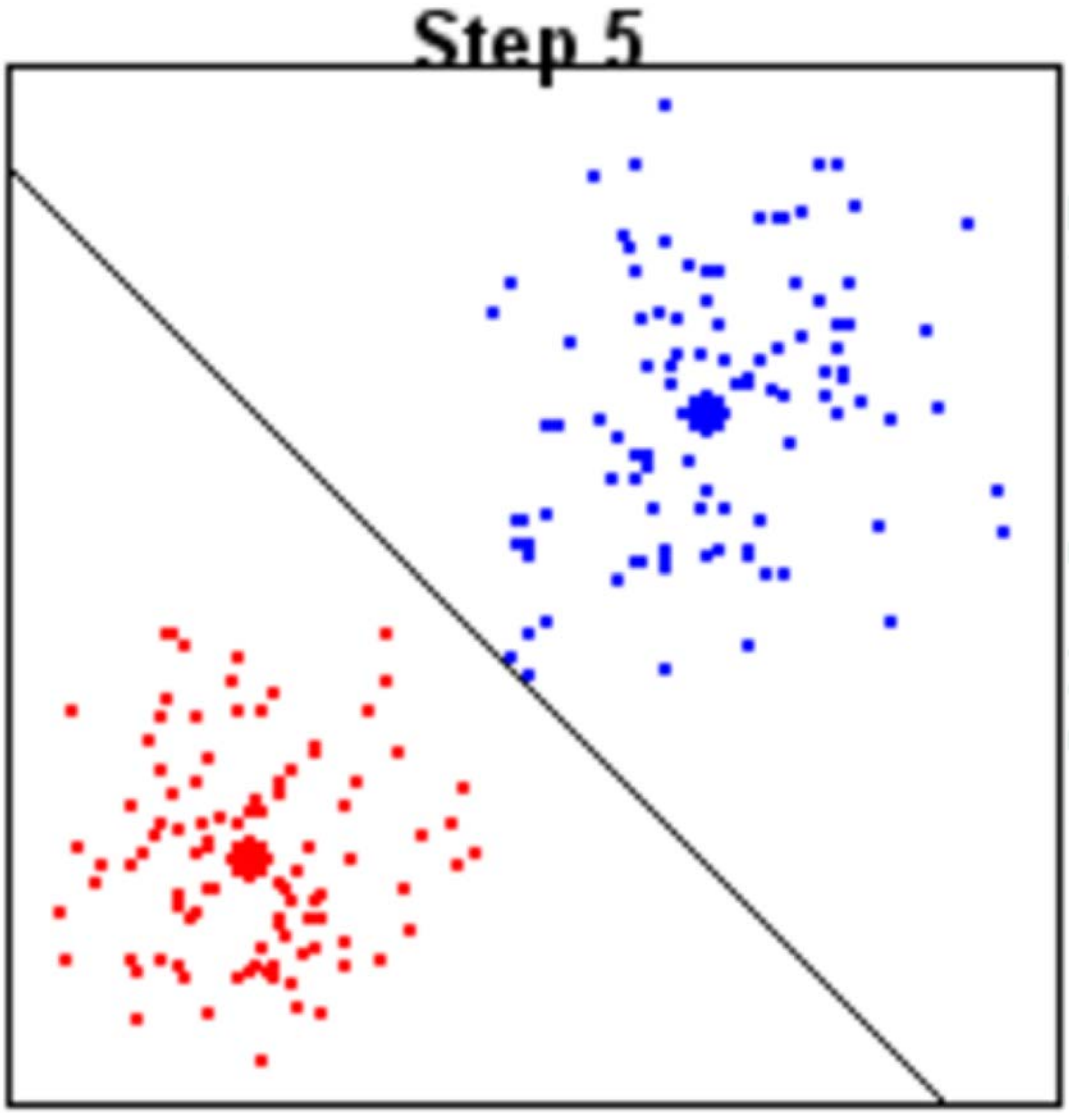
This slide's graphics are from PSU Summer School in Statistics for Astronomers

Recalculate cluster centers



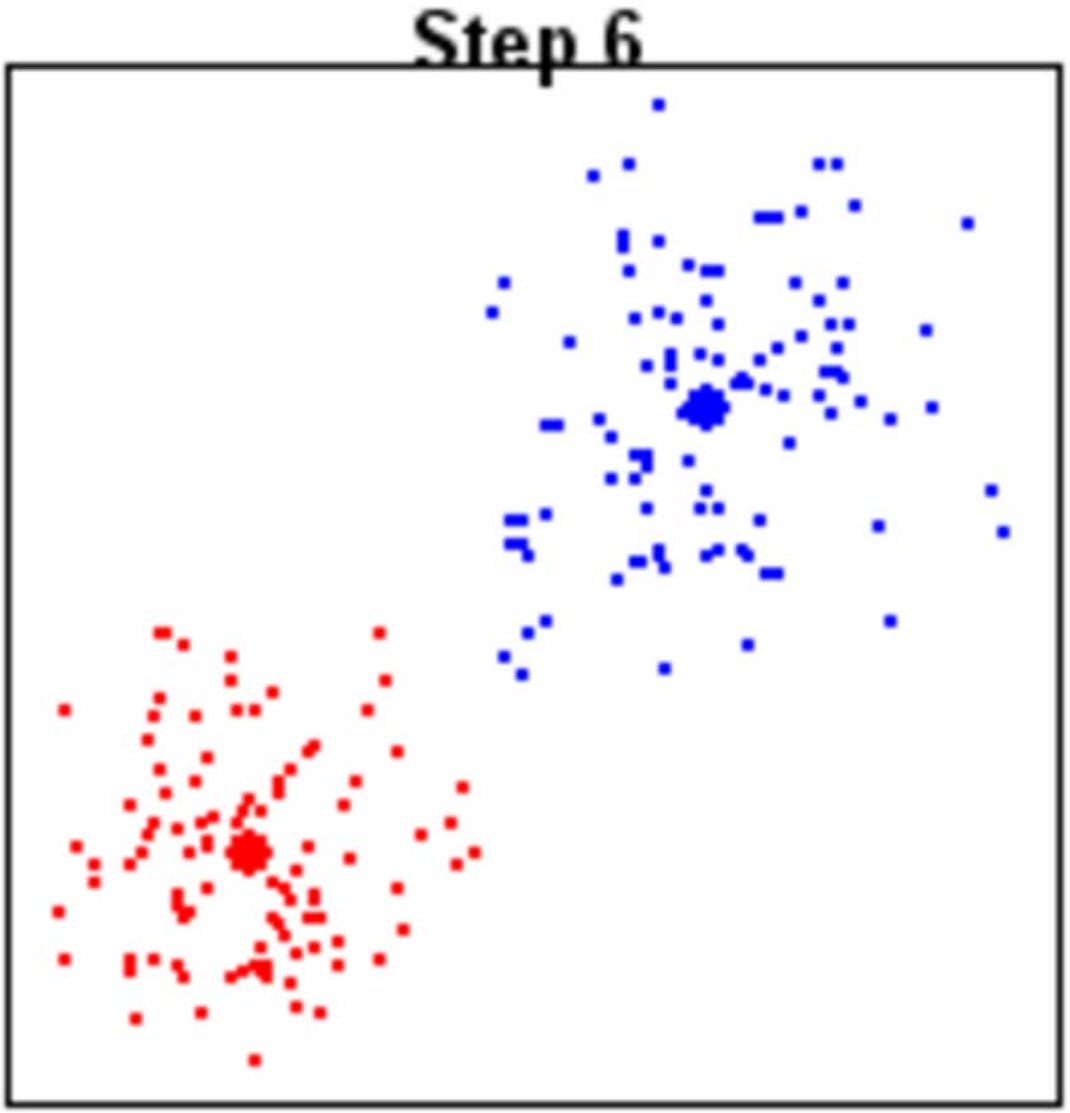
This slide's graphics are from PSU Summer School in Statistics for Astronomers

Reassign points to clusters



This slide's graphics are from PSU Summer School in Statistics for Astronomers

…Until convergence



This slide's graphics are from PSU Summer School in Statistics for Astronomers

K means

for $k = 1$ to K do: # initialize randomly

$\mu_k \leftarrow$ some random location

repeat until converged:

for $j = 1$ to m do:

$y^{(j)} \leftarrow \arg\min_k \left\|\mu_k - \mathbf{x}^{(j)}\right\|$ # A. assign example

for $k = 1$ to K do:

$\mu_k \leftarrow \frac{1}{m_k} \sum_{\mathbf{x}^{(j)} \in k} \mathbf{x}^{(j)}$ # B. re-estimate center

return $\boldsymbol{y}$

Proof of convergence

Only lines A and B change values of μ and y. We can show that both can only decrease J_{clust}.

J_{clust} starts with some value after initialization and is lower bounded by 0.

As y is discrete, there can only be a finite number of steps if each step can only decrease J_{clust}.

Line A: $\left\|\mathbf{x}^{(j)} - \mu(t)\right\| \leq \left\|\mathbf{x}^{(j)} - \mu(t-1)\right\|$

Line B: μ_k calculated by mean, which minimizes MSE.

Decreasing J_{clust}

Line A: Assign each point to its closest cluster

$\rightarrow$ remain/decrease J_{clust}

Line B: For each k, choose μ_k to minimize:

$$J_k = \sum_{x^{(j)} \in S_k} \left|\left|\mathbf{x}^{(j)} - \mu_k\right|\right|^2$$

Let center of points in cluster k be $\bar{x}_k = \frac{1}{m_k} \sum_{x^{(j)} \in S_k} x^{(j)}$

$$J_k = \sum_{\mathbf{x}^{(j)} \in S_k} (\mathbf{x}^{(j)} - \bar{\mathbf{x}}_k + \bar{\mathbf{x}}_k - \mu_k)^2$$

$$= \sum_{\mathbf{x}^{(j)} \in S_k} (\mathbf{x}^{(j)} - \bar{\mathbf{x}}_k)^2 + \sum_{x^{(j)} \in S_k} (\bar{\mathbf{x}}_k - \mu_k)^2 \geq \sum_{\mathbf{x}^{(j)} \in S_k} (\mathbf{x}^{(j)} - \bar{\mathbf{x}})^2$$

Setting $\mu_k \leftarrow \bar{\mathbf{x}}$ $\rightarrow$ remain/decrease J_{clust}

K Means Extensions

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Initialization

… is an issue, but quite surmountable.

Furthest-first heuristic: after picking first point at random, pick all remaining $k-1$ centers to be as far as possible from all previously selected centers, i.e.:

$$m = \arg\max_j \min_{k'<k} \left\|\mathbf{x}^{(j)} - \mu_{k'}\right\|^2$$

Then set $\mu_k = \mathbf{x}^{(j)}$.

Deterministic after the first point selection.

Adding randomness

k-means++*: When selecting μ_k, choose point at random, but in probability proportional to squared distance.

for $k = 2$ to K do:

$$d^{(j)} \leftarrow \min_{k'<k} \left|\left|\mathbf{x}^{(j)} - \mu_{k'}\right|\right|^2, \forall j$$ # compute distances

$$\mathbf{P}: p(j) \leftarrow \frac{d^{(j)}}{\sum_{j=1}^{m} d^{(j)}}, j \in \{1, \dots, m\}$$ # make prob. distribution

$$c \sim \mathbf{P}$$ # pick an example as the center

$$\mu_k \leftarrow \mathbf{x}^{(c)}$$

Run k-means using $\mu_{1..K}$ as the initial centers.

*(Standard initialization in sklearn)

Choosing K

"Should I add another cluster?"

<u>Quick Question</u>: Using J_{clust} as a fitness measure is no good alone because it will always prefer:

1) Only 1 cluster for all points ($K = 1$).

2) The same number of clusters that I'm using now, K.

3) m clusters, one for each point.

Regularizing K means

Augment another term for optimization to ensure value of k is not overly complex. With:

Bayes Information Criterion (BIC): $\arg\min_{K} J_{clust}^{(K)} + K \log n$

Akaike Information Criterion (AIC): $\arg\min_{K} J_{clust}^{(K)} + 2Kn$

Stronger regularizer, especially for high dimensional data

…where n is the dimensionality of $\mathbf{x}$.